



The Singleton Pattern

Human Computer Interaction Research
University of Nevada, Reno



Singleton

☐ Behavioral Patterns

- » strategy
- » observer

☐ Structural Patterns

- » decorator

☐ Creational Patterns

- » factory method
- » abstract factory
- » singleton

problem

“Some classes only need to be instantiated **once** otherwise you get incorrect behavior, overuse of resources and inconsistent results”

Examples



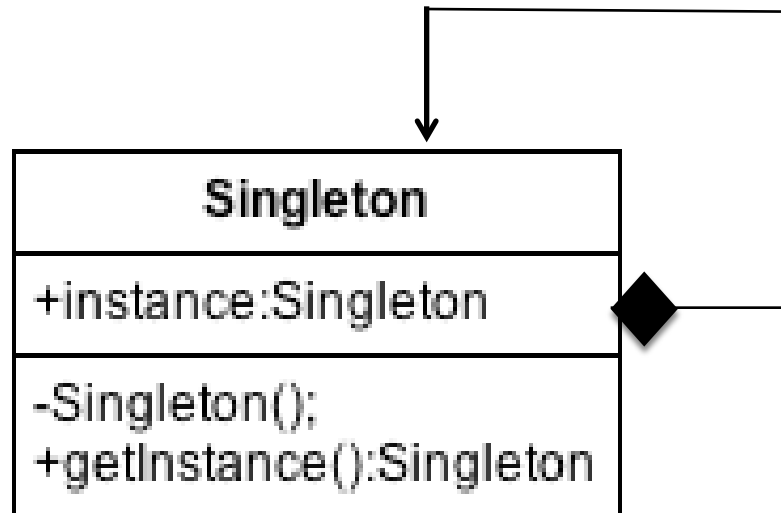
- ☐ Logging
- ☐ Drivers
- ☐ Caching
- ☐ Security
- ☐ Database connection

- ☐ incorrect behavior
- ☐ overuse of resources
- ☐ inconsistent results

Singleton

Singleton ensures a class has only one instance, and provides a global access point to it.

class diagram



Solution

```
public class Singleton {  
    private static Singleton uniqueInstance;
```

```
    private Singleton() {}
```

private so only
singleton can call it

```
    public static Singleton getInstance() {  
        if (uniqueInstance == null) {  
            uniqueInstance = new Singleton();  
        }  
        return uniqueInstance;  
    }  
}
```

Lazy
instantiation

static method e.g.
a CLASS method

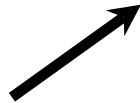
How to instantiate singleton?

`public static Singleton getInstance()`



static method e.g.
a CLASS method

`Singleton var = Singleton.getInstance()`



class name
not a variable

Problem? Not Thread Safe

```
public class Singleton {  
    private static Singleton uniqueInstance;
```

```
    private Singleton() {}
```

```
    public static Singleton getInstance() {
```

```
        if (uniqueInstance == null) {
```

```
            uniqueInstance = new Singleton();
```

```
        }
```

```
        return uniqueInstance;
```

```
    }
```

```
}
```

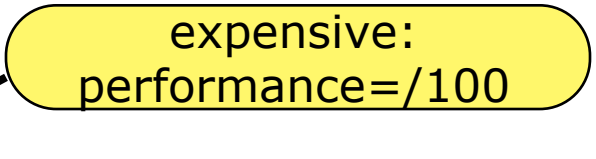
Thread 1

Thread 2

solution: add synchronized

```
public class Singleton {  
    private static Singleton uniqueInstance;  
  
    private Singleton() {}  
  
    public static synchronized Singleton getInstance() {  
        if (uniqueInstance == null) {  
            uniqueInstance = new Singleton();  
        }  
        return uniqueInstance;  
    }  
}
```

expensive:
performance=/100



solution 2: double checked locking

```
public class Singleton {  
    private volatile static Singleton uniqueInstance;
```

```
    private Singleton() {}
```

handles var correctly when
multiple threads init

```
    public static Singleton getInstance() {
```

```
        if (uniqueInstance == null) {
```

```
            synchronized (Singleton.class) {
```

```
                if (uniqueInstance == null) {
```

```
                    uniqueInstance = new Singleton();
```

```
                }
```

```
            }
```

```
        }
```

```
        return uniqueInstance;
```

```
    }
```

```
}
```

only sync
once

doesn't work
in Java < 1.4

volatile

- ❑ The value of this variable will never be cached thread-locally
- ❑ Access to the variable acts as though it is enclosed in a synchronized block, synchronized on itself

Benefits Singleton

- ☐ Ensures you only at most have **one** instance
- ☐ Provides global **access** point
- ☐ Examine your **performance** requirements to determine which implementation of the singleton works best.
- ☐ Some implementations don't work with older version of Java (see book)

Exercise I



Logging

projector

desk

Group 1

Group 2

Group 3

Group 7

Group 8

Group 9

pillar

Group 13

Group 14

Group 18

Group 19

Group 20

Group 24

Group 25

Group 26

Group 27

Group 28

Group 29

Group 4

Group 5

Group 6

Group 10

Group 11

Group 12

Group 15

Group 16

Group 17

Group 21

Group 22

Group 23

pillar

skateboards

door